

Python Core

[Home](#) > [Course](#) > [Python Core](#) > [M3. Data Types](#) > [L1: Data Types Overview](#)[COURSE](#) [PROGRESS](#) [NAVIGATION](#) [DISCUSSION](#)

Navigation

[Expand All](#)

Python Core

- ▼ M0. Introduction 
- ▼ M1. Meet Python
Practical Tasks and Quizzes
- ▼ M2. Development Environment
- ▲ M3. Data Types
Practical Tasks and Quizzes
 - [L0: Module Introduction](#)
 - [L1: Data Types Overview](#)
 - [L2: Numbers](#)
 - [Practical Tasks: Numbers](#)
 - [L3: Boolean and Comparison Operator](#)
 - [L4: Loops](#)
 - [L5: Strings](#)
 - [Practical Task: Strings](#)
 - [L6: Lists](#)
 - [Practical Task: Lists](#)
 - [L7: Tuples](#)
 - [Practical Task: Tuples](#)
 - [L8: Sets](#)
 - [L9: Dictionaries](#)
 - [Practical Task: Dict](#)
 - [Quiz](#)
 - [Final Practical Task](#)
- ▼ M4. Functions
Practical Tasks and Quizzes
- ▼ M5. Modules and Packages
Practical Tasks and Quizzes
- ▼ M6. OOP in Python
Practical Tasks and Quizzes
- ▼ Final Assessment
- ▼ Course Completion

[PREVIOUS](#)[NEXT](#)

L1: Data Types Overview

Lesson Introduction

This lesson will explain concrete data types in Python and describe some basic concepts. You will start from variables. In fact, you don't operate with data directly. You always use it with the help of variables (at least in Python). Then you will review objects. In Python, everything is an object, and you need to know what is behind variables. Finally, operators are used every day and everywhere.

The estimated time: **12 minutes**

Variables and Objects

A variable is a basic term in programming languages. Without them, the program can't do calculations.

In Python, there are no special places in the code for declaring variables. You should just write the name, after that the assignment operator = and the value. In fact, assignment creates the variable:

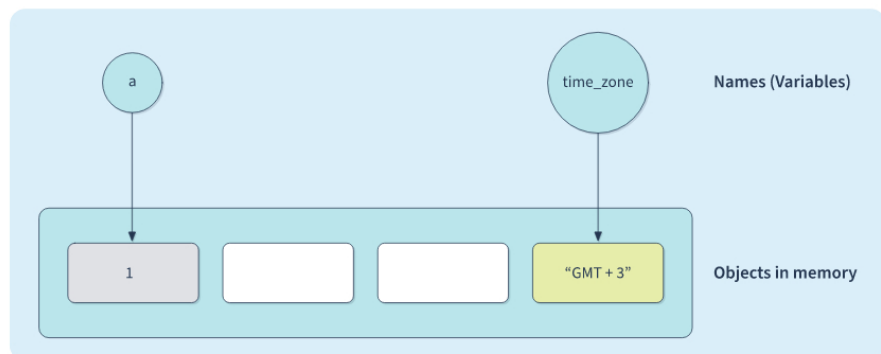
```
a = 1 # There integer data type is used
time_zone = "GMT+3" # There string data type is used
```

After you have created a variable, you can do something with it. For example, you can print its value to the console:

```
print(time_zone) # And you will see ... GMT+3
```

At first, it is convenient to imagine a variable just as a kind of box or storage. You can put something there in your code, then take it, change it and even remove the box if you no longer need it. But to be more precise, from a technical point of view, in Python, a variable is a name attached to a particular object in memory.

You can imagine it approximately like that:



An object is an entity that has a state and behavior, or in other words, attributes, properties, and methods/actions.

An **object** is an instance of some class.

In Python, everything is an object, even classes themselves or functions. It can look strange at the beginning so, don't worry if you feel confused. Right now, you should just note and remember it.

There are a few examples of how you can discover an object nature of Python:

```
a = 10

def foo():
    pass
```

```
class Bar:
    pass

print(type(a))
print(type(foo))
print(type(Bar))
```

As you know, everything is an object in Python, including classes and functions. That means you can invoke a class or function name as a variable and send it as a parameter to another function.

That's why you used the built-in function `type` for getting a type of our objects. You will see in the console:

Overview of Standard Data Types

A variable or better to say a linked object always has an exact type. It can be one of the standard or custom types, such as a class you created. For this reason, let's take a quick look at the standard data types in Python.

Click each heading to see more information.

^ Numeric

Numeric is the data that has a numeric value. There are three different types in Python: integer, floating-point number, and complex number. Also, Boolean is a subtype of an integer. **Integer type** allows you to work with positive or negative whole numbers. **Float type** provides floating-point representation. **Complex** is specified as (real part) + (imaginary part), for example `-4+5j`.

Code

```
a = 10
b = 10.5
c = 8+3j
print(type(a))
print(type(b))
print(type(c))
```

Result

```
<class 'int'>
<class 'float'>
<class 'complex'>
```

You will discover more details and examples in the lesson about numbers.

^ String

String is a text sequence of character data.

Code

```
my_str = "This is string!"
print(type(my_str))
```

Result

```
<class 'str'>
```

You will discover more details and examples in the lesson about strings.

^ List

List is a sequence of the elements inside square brackets `[]`, separated by commas. The number and type of list elements are not limited.

Code

```
some_list = [1, 2.5, True, "Str in a list"]
print(type(some_list))
```

Result

```
<class 'list'>
```

You will discover more details and examples in the lesson about lists.

^ Tuple

Tuple is another type of sequence in Python. It looks like a list but is enclosed in parentheses `()` instead of square brackets `[]`. But there is sufficient internal deference from the list, and you will see it later.

Code

```
my_tuple = (22, 89, True, False)
print(type(my_tuple))
```

Result

```
<class 'tuple'>
```

You will discover more details and examples in the lesson about tuples.

^ Set

Set is an unordered collection of exclusive items (no duplicates) separated by commas inside {}.

Code

```
our_set = {4, 3, 6.6, "Hello"}  
print(type(our_set))
```

Result

```
<class 'set'>
```

You will discover more details and examples in the lesson about sets.

^ Dictionary

Dictionary is a mapping of key-value pairs separated by commas inside {}.

Code

```
another_dict = {"message": "Hi!", 33: True}  
print(type(another_dict))
```

Result

```
<class 'dict'>
```

You will discover more details and examples in the lesson about dictionaries.

Mutable and Immutable Objects

It is important to emphasize one more topic connected to data types because its awareness will help you decrease mistakes that will save your team time.

Click each card to see more information.

In Python, some objects can change after their creation. They are called **mutable objects**. For example, objects whose type is a list or dictionary are mutable.



Immutable Objects

In Python, every object has a unique identifier, and you can use another built-in function `id()` to get it. As you can see in the example below, after changing, the variable `a` has another id. That means that the variable name stays the same. But after changing, it refers to another object in memory.

Code

```
a = 1  
print(id(a))  
  
a += 2  
print(id(a))
```

Result

```
9756224  
9756288
```

In another example with the list, the id of the object is the same after appending a new item in a list.

Code

```
my_list = [1, 2]  
print(id(my_list))  
  
my_list.append(3)  
print(id(my_list))
```

Result

```
22521882389504  
22521882389504
```

It is highly recommended you remember the mutable or immutable types because mutable objects can be a reason for complicated mistakes. For example, you can inadvertently change a parameter inside a function, which will affect the value outside the function. This can be unexpected and confusing.

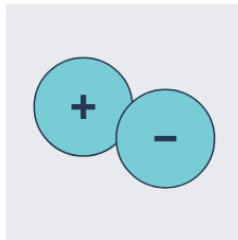
Additional information about this topic you will see later.

Operators and Operands

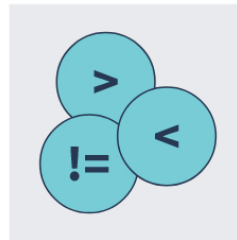
Finally, the last topic of this lesson is operators and operands. You have already used them because you cannot declare, for example, any variables without them.

First, what are **operators** and **operands**? In simple terms, in the expression `2 + 3`, the numbers `2` and `3` are called **operands**, and the `+` sign is the **operator**.

In Python, there are the following operators depending on the number of operands:



Unary Operators



Binary Operators



Ternary Operators

These operators are divided into the following categories:

- **Arithmetic operators** such as `+` (addition), `-` (subtraction), `*` (multiplication), `/` (division), and others.
- Several types of **assignment operators**: `+=` (add and equal), `-=` (subtract and equal), `*=` (multiply and equal), etc.
- **Comparison operators** such as `==` (equal), `!=` (not equal), `>` (greater), `<` (less), etc.
- **Logical operators**: `and` (logical AND), `or` (logical OR), `not` (logical NOT), etc.
- **Membership operators** `in`, `not in` are used to test if a sequence is presented in an object.
- **Bitwise operators** are used to compare (binary) numbers: `&` (bitwise AND), `|` (bitwise OR), `^` (bitwise XOR), etc.

You will find more information in the next lessons.

Lesson Summary

A **variable** is named storage for value in the memory, used in the program code.

An **object** is an entity that has a state and behavior, or in other words, attributes, properties, and methods/actions. An **object** is an instance of some class. In Python, everything is an object, even classes themselves or functions.

In Python, some of the objects can be changed after their creation. They are called **mutable objects**. On the other hand, there are **immutable objects** that can't be changed after creation.

[< PREVIOUS](#)

[NEXT >](#)